IBM Innovation Centre for Education

NCS NihonCommunicationSolutions

Yenepoya

Project – Low Level Design

on

Cloud Forensic Automation For Healthcare Breaches

Student Name: Sreelakshmi Sreejith

Industry Mentor:Mr Shashank | Guided by:Ms Divya Shetty

Table of Contents

1. Introduction 1
    1.1 Purpose 1
    1.2 Scope 2
    1.3 Relationship with HLD 2
2. System Architecture 3
    2.1 Project Structure 4
    2.2 Module Overview 5
3. Module Design 6
    3.1 Scanner Module 6
    3.2 Metadata Module 7
    3.3 Hashing Module 7
    3.4 Forensic Service Module 8
    3.5 Report Generator 8
4. Process and Algorithms 9
    4.1 Forensic Scan Workflow 9
    4.2 Input Validation Logic 9
5. Data Design 10
    5.1 Data Model 10
    5.2 Data Storage and Access 10
6. API Design 11
    6.1 Scan API 11
    6.2 Report Download API 11
7. User Interface Design 12
    7.1 Dashboard Overview 12
    7.2 Visualization and Interaction 12
8. Error Handling and Logging 13
9. Testing 13
10. Deployment 14
11. References 14

1. Introduction

The Low Level Design (LLD) document provides a detailed technical description of the MedTrace Healthcare Cloud Forensics System. It explains how each component of the system is implemented internally, including modules, functions, data handling, and processing logic. While the High Level Design (HLD) focuses on the overall architecture, this document goes deeper into the internal working of the system.

MedTrace Forensics is developed as a web-based application using Flask, designed to investigate healthcare data breaches. The system scans directories containing healthcare data, analyzes file artifacts, verifies integrity using cryptographic hashing, and generates structured evidence reports. This LLD describes how these operations are carried out step by step at the implementation level.

The document serves as a technical guide for developers, testers, and evaluators. It ensures that all components of the system are clearly defined and can be implemented consistently. It also helps in understanding how different modules interact with each other to perform forensic analysis.

In addition, this document outlines the flow of data between components and highlights key design decisions taken during development. It provides clarity on system behavior under different conditions and helps in identifying potential improvements or optimizations. The LLD also supports effective testing, debugging, and future enhancements by providing a clear and structured implementation reference.

1.1 Purpose

The purpose of this Low Level Design document is to provide a clear and detailed explanation of how the MedTrace Forensics system is implemented. It acts as a bridge between the conceptual design (HLD) and the actual development of the system.

This document helps in:

- Defining the internal structure of each module
- Explaining how data is processed and transformed
- Describing algorithms used for scanning and analysis
- Providing clarity on API behavior and system responses
- Supporting developers during coding and debugging
- Assisting testers in validating system functionality

By providing implementation-level details, the LLD ensures that the system can be developed accurately and efficiently without ambiguity.

1.2 Scope

This document covers all technical aspects related to the internal implementation of the MedTrace Forensics system. It focuses on how the system operates rather than what the system does.

The scope includes:

- Detailed module design (scanner, metadata extractor, hashing engine, report generator)
- Step-by-step algorithms for forensic processing
- Data structures and formats used within the system
- API request and response handling
- Error handling and logging mechanisms
- Testing strategies and validation methods
- System configuration and execution details

The document is limited to software-level implementation and does not include hardware setup or deployment infrastructure in detail. It is intended primarily for development and academic evaluation purposes.

1.3 Relationship with HLD

The Low Level Design (LLD) is directly derived from the High Level Design (HLD) of the system. While the HLD provides an overview of system architecture and major components, the LLD provides detailed specifications for implementing those components.

Each section of the HLD is expanded in this document to include technical details such as function definitions, data formats, and processing logic. For example, the components defined in the HLD (such as scanner, hash engine, and reporting module) are described here with their internal workflows and responsibilities.

The relationship between HLD and LLD ensures consistency in system design and implementation. It allows developers to translate high-level concepts into working code while maintaining alignment with the original system architecture.

The LLD also supports testing and maintenance by providing clear documentation of how each part of the system functions. This makes it easier to identify issues, update modules, and extend the system in the future.Overall, the LLD complements the HLD by transforming design concepts into practical implementation details, ensuring a complete and structured development process.

2. System Architecture

The MedTrace Healthcare Cloud Forensics System is designed using a modular and layered architecture to ensure clarity, scalability, and ease of maintenance. The system follows a web-based client-server model, where the frontend handles user interaction and the backend performs processing and analysis. This separation ensures efficient operation and better organization of system components.

The frontend is responsible for displaying the dashboard, charts, alerts, and scan results. It is built using HTML, CSS, and JavaScript, with Chart.js used for visualization. Users interact with the system through this interface to initiate scans, view results, and download reports. The interface is designed to be simple and intuitive, allowing investigators to easily understand and analyze forensic results.

The backend is implemented using Flask, which handles user requests, processes scan operations, and manages communication between modules. It ensures that data is processed correctly and returned to the frontend in a structured format. The backend acts as the core of the system, coordinating all processing tasks efficiently.

The system is divided into modules such as scanning, metadata extraction, hashing, classification, and reporting. The scanner retrieves files, the metadata module extracts file details, the hashing module generates cryptographic hashes, and the classification module determines risk levels. The report generator compiles all results into a CSV file for further analysis.

Each module performs a specific function and works together in a structured workflow. This modular design improves maintainability, simplifies debugging, and allows future enhancements without affecting the overall system. It also ensures that errors in one module do not impact others.

The architecture supports smooth data flow between components, ensuring that outputs from one stage are correctly passed to the next. This improves consistency and accuracy in forensic analysis.

Overall, the system architecture provides a reliable and efficient framework for forensic investigation. It ensures flexibility, scalability, and ease of use, making it suitable for handling healthcare data securely and effectively.

2.1 Project Structure

The project follows a well-structured directory format that separates backend logic, frontend interface, and data storage. This improves readability, maintainability, and overall development efficiency.

The main folders and files are organized as follows:

```
medtrace_forensics/
│
├── app.py                # Main Flask application (backend entry point)
├── modules/              # Core processing modules
│   ├── scanner.py        # File scanning module
│   ├── metadata.py       # Metadata extraction module
│   ├── hasher.py         # Hash generation module
│   ├── forensic_service.py # Main processing and classification logic
│   ├── reporter.py       # Report generation module
│
├── templates/            # HTML templates (frontend UI)
│   ├── index.html
│   ├── login.html
│
├── static/               # Static assets (CSS, JavaScript)
│   ├── style.css
│   ├── script.js
│
├── reports/              # Generated forensic reports (CSV)
├── logs/                 # System and scan logs
├── sample_healthcare_data/ # Sample dataset for testing
└── requirements.txt      # Project dependencies
```

This structure ensures a clear separation of responsibilities between different components of the system. It allows developers to easily locate files, update modules, and maintain the application efficiently. The modular organization also supports scalability and future enhancements.

2.2 Module Overview

The system is divided into multiple modules, each responsible for a specific function within the forensic workflow. These modules work together to process data and generate results.

• Scanner Module:
This module scans the given directory and collects all valid file paths. It ensures that only relevant files are included in the analysis.

• Metadata Module:
This module extracts important information from each file, such as file name, size, timestamps, and permissions. This data is used for further analysis.

• Hashing Module:
This module generates cryptographic hash values (SHA-256) for each file. These hashes help verify file integrity and detect any modifications.

• Forensic Service Module:
This acts as the main processing unit of the system. It combines scanning, metadata extraction, and hashing, and performs risk classification based on file modification time.

• Report Generator Module:
This module generates a CSV report containing all processed file details, including metadata, hash values, and risk classification.

All modules are designed to work independently but are integrated through the backend. This modular approach improves system maintainability and allows easy updates or enhancements in the future. It also ensures that each module can be tested separately without affecting the entire system.

The clear division of modules helps in better understanding of system functionality and simplifies debugging during development. It also allows developers to replace or upgrade individual modules without modifying the whole system.

The interaction between modules follows a structured sequence, where the output of one module becomes the input for the next. This ensures consistency and accuracy in the processing of forensic data.

Overall, the system architecture ensures smooth data flow, efficient processing, and a clear separation of responsibilities across different components.

3. Module Design

The MedTrace Healthcare Cloud Forensics System is designed using a modular approach, where each module is responsible for a specific function within the forensic workflow. This design improves clarity, maintainability, and scalability of the system.

Each module performs a well-defined task such as scanning files, extracting metadata, generating hash values, classifying risk, and generating reports. These modules work together in a structured sequence to complete the forensic analysis process.

The modular design ensures that changes in one module do not affect other parts of the system. It also allows developers to test, debug, and enhance individual modules independently.

Overall, the module-based architecture improves system efficiency and provides a clear structure for implementation and future enhancements.

3.1 Scanner Module

The Scanner Module is responsible for scanning the given directory and retrieving all valid file paths for analysis. It acts as the starting point of the forensic workflow.

This module uses directory traversal techniques to explore all files within the specified path. It ensures that only relevant files are selected for processing, while unnecessary or system files are ignored.

The scanner reads the directory structure recursively, allowing it to access files in nested folders. It collects the absolute path of each file and passes it to the next module for further processing.

The module also handles edge cases such as empty directories, invalid paths, and restricted files. If a directory does not exist, an appropriate error is generated. If a file cannot be accessed, it is skipped and logged without stopping the entire scan.

The output of this module is a list of file paths, which is used as input for the metadata extraction module. This ensures a smooth flow of data between modules.

The Scanner Module is designed to be efficient and reliable, ensuring that all relevant files are identified correctly. It plays a critical role in ensuring that the forensic analysis covers all necessary data.

The module is designed to handle large directories efficiently without affecting system performance. It ensures that the scanning process is fast, reliable, and suitable for forensic analysis.

3.2 Metadata Module

The Metadata Module is responsible for extracting important information from each file identified by the scanner module. It plays a key role in analyzing file properties that are essential for forensic investigation.

This module collects details such as file name, file path, file size, creation time, modification time, and access time. These attributes help in understanding the history and state of each file.

The module uses system-level functions to access file information and converts raw data into a structured format. This structured data is easier to process and is passed to the next module for further analysis.

One of the most important attributes extracted is the modification time, which is used later to determine whether a file is suspicious or not. This helps in identifying files that were changed after the breach event.

The Metadata Module also checks for anomalies such as zero-byte files or unusually large files. These anomalies may indicate suspicious activities like file deletion or data manipulation.

The extracted metadata is stored in a structured format such as a dictionary, which allows easy access and processing. This ensures smooth integration with other modules in the system.

3.3 Hashing Module

The Hashing Module is responsible for generating cryptographic hash values for each file. It ensures data integrity by verifying whether a file has been modified or tampered with.

This module uses hashing algorithms such as SHA-256 to generate a unique hash value for every file. Even a small change in file content results in a completely different hash, making it useful for detecting modifications.

The module reads files in chunks to handle large files efficiently without consuming excessive memory. This improves system performance during the hashing process.

The generated hash values are added to the file's metadata and passed to the next stage of processing. These values are also included in the final report for verification purposes.

The Hashing Module ensures accuracy and reliability in forensic analysis by providing strong proof of file integrity.

The module is designed to handle different file types efficiently without affecting performance. It plays an important role in ensuring that all files are verified consistently during the forensic analysis process.

3.4 Forensic Service Module

The Forensic Service Module acts as the core processing unit of the system. It integrates the outputs from the scanner, metadata, and hashing modules and performs the main forensic analysis.

This module receives the list of file paths from the scanner module and processes each file sequentially. It combines metadata information and hash values to create a complete record for every file.

The primary function of this module is to classify files based on risk level. It compares the file modification time with the predefined breach date. If a file is modified after the breach date, it is marked as High Risk; otherwise, it is classified as Low Risk.

The module also organizes all processed data into a structured format, ensuring consistency and ease of understanding. It manages the workflow between different modules and ensures that data flows smoothly from one stage to another.

Additionally, it handles errors efficiently, allowing the system to continue processing even if certain files encounter issues. This improves system reliability and ensures uninterrupted analysis.

3.5 Report Generator Module

The Report Generator Module is responsible for creating a structured output of all processed forensic data. It compiles the results from the forensic service module into a CSV file.

The module includes details such as file metadata, hash values, risk levels, and classification results in the report. This provides a complete summary of the forensic analysis.

The generated report is stored in the system and can be downloaded by the user for further investigation or documentation. The CSV format ensures compatibility and ease of use.

The module ensures that all data is written accurately and in a consistent format. It also handles file creation and storage efficiently.

Overall, the Report Generator Module plays an important role in documenting the results of the forensic analysis and supporting further decision-making.

The module also ensures that the report is generated in a clear and organized format for easy interpretation. This helps investigators quickly analyze results and make informed decisions during the forensic process.

4. Process and Algorithms

This section describes the workflow and algorithms used in the MedTrace Healthcare Cloud Forensics System. It explains how data is processed step by step, from user input to final report generation.

The system follows a structured sequence where each module performs a specific task, and the output of one module becomes the input for the next, ensuring smooth data flow.

This structured approach helps maintain accuracy and consistency throughout the forensic analysis process. It also simplifies understanding of system behavior and improves overall processing efficiency. The clear workflow ensures that each stage of analysis is completed correctly before moving to the next step.

4.1 Forensic Scan Workflow

The forensic scan workflow defines the process followed during a scan operation.

The process begins when the user enters a directory path and initiates a scan. The system validates the input and then scans the directory using the scanner module. All valid file paths are collected and passed to the metadata module.

The metadata module extracts file details such as name, size, and timestamps, which are then sent to the hashing module to generate cryptographic hash values.

The forensic service module then processes the data by combining metadata and hash values and classifies files as High Risk or Low Risk based on modification time.

Finally, the report generator module compiles the results into a CSV file. The results are displayed on the dashboard and made available for download.

4.2 Input Validation Logic

The input validation logic ensures that the directory path provided by the user is valid and safe.

The system checks whether the path exists and is a valid directory. If invalid, an error message is shown and the scan is not started.

It also ensures that the input does not contain unsafe patterns that may lead to security issues, preventing unauthorized access.

If the input is valid, the system proceeds with scanning; otherwise, it stops execution and informs the user.

5. Data Design

The Data Design section describes how data is structured, stored, and processed within the MedTrace Healthcare Cloud Forensics System. The system does not rely on a traditional database; instead, it uses in-memory data structures and file-based storage for efficient processing.

Data is collected during the scanning process and passed through different modules such as metadata extraction, hashing, and classification. Each module processes the data and enriches it with additional information, ensuring a structured flow throughout the system.

The final processed data is stored temporarily in memory and then exported as a CSV report. This approach ensures fast execution and avoids the overhead of database management.

5.1 Data Model

The system uses a simple and structured data model to represent each file as a record. Each record contains important attributes required for forensic analysis.

The main attributes include file name, file path, file size, creation time, modification time, hash value, and risk level. These attributes help in identifying suspicious files and understanding their state.

Each file record is stored in a structured format such as a dictionary or object. This allows easy access, modification, and transfer of data between modules.

The data model is designed to be simple yet effective, ensuring that all relevant information is captured without unnecessary complexity.

5.2 Data Storage and Access

The system stores data temporarily in memory during the scanning process. This allows fast processing and easy data transfer between modules.

Once processing is complete, the data is written to a CSV file in the reports directory. This file acts as the final output of the forensic analysis and can be downloaded by the user.

The system accesses files using standard file system operations and reads data efficiently without loading entire files into memory.

This approach ensures efficient data handling while maintaining simplicity and reliability in the system design.

6. API Design

The API Design defines how the frontend communicates with the backend in the MedTrace Healthcare Cloud Forensics System. The system uses REST-based APIs to handle user requests and return processed data.

These APIs ensure smooth interaction between the user interface and backend processing modules. They are designed to be simple, efficient, and secure. The APIs follow a structured request-response format, allowing consistent communication between different components of the system.

The backend processes incoming requests, performs the required operations, and sends responses in a structured format such as JSON or file output. This approach improves system reliability and makes integration easier.

6.1 Scan API

The Scan API is used to initiate the forensic scanning process.

When the user provides a directory path and starts the scan, a POST request is sent to the backend. The backend validates the input and begins processing using the scanner, metadata, and hashing modules.

The API returns structured JSON data containing file details, hash values, and risk classification. This data is displayed on the dashboard for analysis.

The API also handles errors such as invalid paths or inaccessible files and returns appropriate messages to the frontend. This ensures that users are informed about issues during the scanning process.

The Scan API is designed to handle multiple files efficiently and ensures that the response is generated in a consistent format.

6.2 Report Download API

The Report Download API allows users to download the generated forensic report.

Once the scan is completed, the system creates a CSV file containing all processed data. The user can download this file through a GET request.

This API ensures easy access to the report for further investigation and documentation. The downloaded report contains complete details required for forensic analysis.

The API ensures that the file is securely accessed and only available after successful scan completion. It also supports quick retrieval of reports without affecting system performance.

7. User Interface Design

The User Interface provides an interactive dashboard for investigators to perform forensic analysis. It is designed to be simple, responsive, and easy to use.

The interface allows users to initiate scans, view results, monitor system activity, and download reports. It ensures that all features are easily accessible and organized in a structured manner.

The UI is developed using HTML, CSS, and JavaScript, providing a smooth and responsive experience across different devices. Proper layout and alignment are maintained to ensure readability and ease of navigation.

The design focuses on clarity, ensuring that important information such as risk levels and alerts are highlighted clearly for quick understanding. This helps investigators quickly identify suspicious activities.

7.1 Dashboard Overview

The dashboard consists of multiple sections such as Command Center, Threat Visualizer, Live Monitor, Alert Zone, and Control Panel.

Each section provides specific functionality. The Command Center is used to initiate scans, the Threat Visualizer displays analysis results, the Live Monitor shows real-time logs, the Alert Zone highlights high-risk files, and the Control Panel provides system controls.

These sections are organized in a user-friendly layout, allowing users to switch between functionalities easily. This improves efficiency and reduces complexity during investigation.

7.2 Visualization and Interaction

The system uses charts and tables to display results in a clear and understandable format. Chart.js is used to visualize risk levels and file distribution.

Tables are used to present detailed file information such as metadata, hash values, and classification results. This combination of charts and tables helps users understand both summary and detailed data.

Users can interact with the dashboard to explore results, monitor logs, and access reports. Interactive elements such as buttons, filters, and real-time updates improve usability.

The interface also provides immediate feedback during scanning operations, helping users track progress. This enhances user experience and makes the system more efficient for real-time forensic analysis.

8. Error Handling and Logging

The system includes mechanisms to handle errors and maintain logs during execution.

Errors such as invalid directory paths, file access issues, or missing files are handled gracefully without stopping the entire process. The system provides appropriate error messages to the user, ensuring clarity and preventing confusion during operation.

The system is designed to continue processing even if certain files encounter errors. This ensures that the overall scan is not interrupted and that maximum data is analyzed.

Logging is used to record system activities and scan progress. It captures important events such as scan initiation, file processing status, errors encountered, and completion of tasks. This helps in debugging and tracking system behavior during forensic analysis.

Logs are stored in a structured format, making it easier to review and analyze system activity. They also help developers identify issues and improve system performance.

Overall, the error handling and logging mechanisms improve system reliability, maintain stability during execution, and ensure smooth operation of the application.

9. Testing

Testing is performed to ensure that the system functions correctly and produces accurate results.

Different test cases are used to validate scanning, metadata extraction, hashing, and classification. The system is tested with sample healthcare datasets to verify its performance and correctness.

Both functional testing and basic validation testing are carried out to ensure that each module works as expected. Inputs such as valid and invalid directory paths are tested to check system behavior under different conditions.

The system is also tested for handling edge cases, such as empty directories, restricted files, and large datasets. This ensures that the system remains stable under various scenarios.

Testing helps identify errors, improve performance, and ensure that the system meets its requirements. It also ensures that the system produces consistent and reliable results during forensic analysis.

10. Deployment

The MedTrace Healthcare Cloud Forensics System is deployed as a local web-based application, enabling easy execution, testing, and maintenance without requiring complex infrastructure. The system is developed using Python and the Flask framework, which together provide a lightweight and efficient environment for backend processing and web communication.

To run the application, users must install the required dependencies listed in the requirements.txt file. These dependencies can be installed using standard package management tools such as pip, ensuring a simple and consistent setup process across different systems. A virtual environment can also be used to isolate dependencies and avoid conflicts with other projects. Once the installation is complete, the application server is started using the Flask development server.

After launching the server, the system becomes accessible through a web browser via a localhost address (typically http://127.0.0.1:5000). The frontend interface communicates with backend APIs to perform forensic operations such as directory scanning, file integrity verification, anomaly detection, and report generation. This local deployment approach ensures quick setup, minimal configuration, improved performance, and ease of debugging during development.

The project structure is modular and well-organized, supporting smooth deployment and execution. Separate folders for templates, static files, and backend logic improve maintainability and readability. Developers can easily modify or extend specific components without affecting the overall system functionality, making the application scalable and developer-friendly.

Additionally, basic security measures such as input validation and controlled file access are implemented to ensure safe execution in the local environment. Logging mechanisms can also be integrated to track system activity and support forensic auditing during investigations.

The deployment process is platform-independent, allowing the system to run on operating systems such as Windows, Linux, and macOS. This flexibility ensures that users can deploy and test the application in different environments without compatibility issues.

Furthermore, the current deployment model serves as a foundation for future enhancements. The system can be deployed on cloud platforms such as Amazon Web Services, Microsoft Azure, or Google Cloud Platform to enable remote access and scalability. It can also be containerized using tools like Docker for consistent deployment across different environments.

In addition, future deployment can include integration with database servers, load balancing techniques, and secure communication protocols such as HTTPS to improve performance and security in production environments.

In conclusion, the deployment strategy ensures simplicity, flexibility, and efficiency while also providing a clear path for scaling the system into a fully functional real-world forensic solution.

11. References

No.	Resource	Type	Description
1	Flask Documentation	Official Docs	Used for backend development and API handling
2	Python hashlib Library	Library Docs	Used for hashing and file integrity verification
3	Chart.js Documentation	Official Docs	Used for visualizing forensic data
4	National Institute of Standards and Technology (NIST) Digital Forensics Guidelines	Standard	Provided forensic investigation best practices
5	IBM Cybersecurity Resources	Industry Resource	Used for real-world security insights
6	Flask RESTful	Framework Extension	Used for building RESTful APIs
7	GitHub	Version Control	Used for managing and storing project code
8	Visual Studio Code	Development Tool	Used for coding and debugging
9	Computer Security: Principles and Practice	Textbook	Provided cybersecurity concepts and theory
10	Digital Forensics and Incident Response	Textbook	Used for forensic methodologies and techniques